Seqcore: a vectorized Python library for batch biological sequence analysis

Pritam Kumar Panda*[1]

[1]Stanford University, Stanford, CA, USA

Abstract

Motivation. Much routine sequence analysis is elementwise: computing GC content, reverse complements, translations or k-mer spectra applies one small operation to every residue. Python libraries commonly express these as a loop over sequence objects, so runtime is set by interpreter dispatch rather than by the arithmetic, and the cost grows with the number of sequences even though the operation itself does not become harder.

Results. We present **Seqcore**, a library that stores a batch of sequences as a single padded ordinal matrix with an accompanying length vector, so that batch-wide operations become individual NumPy calls whose cost is independent of how many sequences are present. On this representation we build masked reductions for per-sequence statistics, a table-driven translation that never materializes a Python string, an integer k-mer encoding with an alphabet-bound dispatch rule, and an anti-diagonal wavefront formulation of pairwise dynamic programming that reduces interpreter-level iteration from $O(mn)$ to $O(m+n)$. On 100,000 sequences of 1000 bp, Seqcore computes GC content in 33 ms (3.0 Gbase s^{-1}; 37.5× Biopython) and translates in 178 ms (561 Mbase s^{-1}; 28.4× Biopython). We also report the regimes where Seqcore is *not* the right tool: reverse complement sits at 0.84× Biopython, and pairwise alignment is 17× slower than Biopython's compiled aligner. We additionally measure what a GPU backend would be worth, and find that per-call host transfer, not arithmetic, would govern the outcome. Every vectorized routine is pinned to a naive scalar reference by randomized differential tests.

Scope. Beyond sequence operations we apply the same analysis to Seqcore's molecule, structure and phylogenetics modules, where the limiting costs turn out to be repeated parsing and missing spatial indices rather than data layout, and where the appropriate target for a wrapper around an established library is parity rather than advantage.

Availability. Seqcore is available under the MIT licence at `https://github.com/pritampanda15/Seqcore`. The results reported here are from version 0.5.0 (`pip install seqcore>=0.5.0`). Benchmarks, raw results and figure-generation code are included in the repository.

Keywords: sequence analysis; vectorization; NumPy; dynamic programming; bioinformatics software; reproducible benchmarking

1 Introduction

Array programming [2] was designed for exactly the kind of work that dominates routine sequence analysis. Computing GC content over a set of reads, taking reverse complements, translating open reading frames, or counting k-mers all apply the same small operation uniformly across residues, with no data-dependent control flow. Such operations map naturally onto a single vectorized call over a dense numeric array.

In practice, Python bioinformatics libraries usually do not express them that way. Sequences are modelled as objects, and a batch is a list of those objects, so an operation over a batch becomes a loop. Each iteration is individually cheap, but the aggregate cost scales with the number of sequences and is dominated by interpreter work — attribute lookup, temporary allocation, function-call overhead — rather than by the residue arithmetic that the user actually asked for. The effect is invisible at the scale of a handful of sequences and substantial at the scale at which these libraries are typically used.

This pattern also has a more severe form in dynamic programming. A direct transcription of the Needleman–Wunsch recurrence [3] performs one Python loop iteration and several scalar array accesses per matrix cell, so the interpreter cost is $O(mn)$ — the access pattern for which NumPy is least efficient, applied to the largest number of operations in the library.

*Correspondence: `pritam@stanford.edu`

Seqcore is built around the alternative. A batch of sequences is stored once as a padded matrix of small integers together with a vector of true lengths, and every operation is expressed as array operations over that matrix. This paper describes the representation (Section 2.1), the four techniques built on it (Sections 2.2–2.6), the differential testing that establishes their correctness (Section 2.7), and a benchmark comparison against Biopython [1] and idiomatic pure Python (Section 3).

We report the comparison in both directions. Vectorization pays when work amortizes across a batch; it does not pay when a compiled per-element implementation already exists, or when intermediate results fail to collapse. Both situations occur in Seqcore, and Section 3.6 states where they are and what users should do instead.

2 Design

2.1 Array representation

Seqcore represents a batch of n sequences as a matrix $D \in \{0, \ldots, 4\}^{n \times W}$ of unsigned 8-bit integers, where $W = \max_i \ell_i$, together with a length vector $\ell \in \mathbb{N}^n$ (Figure 1a). Nucleotides map to ordinals A$\to$0, C$\to$1, G$\to$2, T/U$\to$3, N$\to$4; amino acids use an analogous 23-symbol alphabet. Entries beyond ℓ_i in row i are padding and carry no meaning.

Two properties of this layout govern everything that follows. First, any elementwise operation on residues becomes an operation on D, expressible as a constant number of NumPy calls regardless of n. Second, correctness on ragged batches reduces to a single recurring question — whether padding is excluded — which is answered once by the boolean mask $M_{ij} = [\, j < \ell_i \,]$ rather than separately inside every function. When all lengths are equal the mask is unnecessary, and Seqcore skips it; this uniform case is common enough to be worth the special case.

The trade-off is memory. A batch whose length distribution is highly skewed pads to its longest member, so nW can substantially exceed $\sum_i \ell_i$. For read-scale data, where lengths are near-uniform, this overhead is small; for mixed short and very long records it is not, and streaming (`read_stream`) exists to bound it.

2.2 Batched encoding

Because the ASCII-to-ordinal map is a 128-entry lookup table, an entire batch can be translated in a single gather over one concatenated buffer:

```
flat = table[np.frombuffer("".join(sequences).encode("ascii"), dtype=np.uint8)]
```

For uniform lengths the result reshapes directly to (n, W). For ragged batches, the flat codes are scattered into the padded matrix using row and column indices derived from a cumulative sum of ℓ — again a constant number of NumPy calls. The only per-sequence work remaining is the concatenation itself, which executes in C.

2.3 Masked reductions

Per-sequence statistics are computed as masked two-dimensional reductions rather than as loops over rows. For GC content,

$$g_i \;=\; \frac{100}{\ell_i} \sum_j \big[\, D_{ij} \in \{1, 2\} \,\big]\, M_{ij}, \tag{1}$$

which is three NumPy calls in total. Reverse complement is handled in the same spirit: the reversal is expressed as a strided view (uniform lengths) or a single `take_along_axis` gather (ragged lengths), and the complement table is applied to that result, so the batch is traversed once rather than twice.

2.4 Table-driven translation

The ordinal encoding makes the genetic code directly addressable. Since each nucleotide is an integer in $[0, 5)$, the codon at offset t in reading frame f maps to the index

$$c_{it} \;=\; 25\, D_{i,f+3t} \;+\; 5\, D_{i,f+3t+1} \;+\; D_{i,f+3t+2} \;\in\; [0, 125), \tag{2}$$

and a precomputed table $T \in [0, 23)^{125}$ maps every such index to an amino acid ordinal. The table covers all 125 indices, including the 61 containing `N`, which resolve to the unknown-residue symbol `X`; no codon requires a fallback path. Because T's outputs are already valid symbols in the protein alphabet, $T[c]$ is used directly as the encoded matrix of the resulting protein array, and no sequence is materialized as a Python string at any stage. Early termination at the first stop codon, when requested, is an `argmax` over a boolean matrix followed by an elementwise minimum against ℓ, so it does not reintroduce a loop.

2.5 Anti-diagonal wavefront alignment

In the alignment matrix each cell (i, j) depends only on $(i-1, j-1)$, $(i-1, j)$ and $(i, j-1)$. Under the anti-diagonal index $d = i + j$ those predecessors lie on diagonals $d-2$ and $d-1$, so all cells sharing a value of d are mutually independent (Figure 1b) and can be evaluated in one vector operation:

$$S_{i,d-i} \;=\; \max\Big(S_{i-1,d-i-1} + \sigma_{i,d-i},\; S_{i-1,d-i} + \gamma,\; S_{i,d-i-1} + \gamma\Big), \tag{3}$$

with σ the substitution score and γ the gap penalty, clamped below at zero in the local (Smith–Waterman [4]) case. Iterating over d rather than over (i, j) reduces the interpreted iteration count from mn to $m + n$ while leaving the recurrence, and hence the resulting matrix, unchanged. The asymptotic work remains $\Theta(mn)$; only the constant factor changes, and it changes more the larger the problem is. Traceback is left as scalar code, since it visits only $O(m + n)$ cells.

This is the same dependence structure that SIMD aligners exploit [5, 6]; here it is used to obtain vector parallelism from NumPy rather than from vector registers directly.

2.6 Integer k-mer counting and its dispatch rule

Since each nucleotide is an ordinal in $[0, 5)$, a k-mer folds into a base-5 integer,

$$\kappa_{it} \;=\; \sum_{u=0}^{k-1} 5^{\,k-1-u}\, D_{i,t+u}, \tag{4}$$

computed by k rolling passes over the matrix and fitting in a signed 64-bit integer for $k \le 26$. Counting then becomes `np.unique` over integers, and only the *distinct* k-mers are decoded back into strings.

This is a win only when k-mers actually collapse onto one another. When nearly every window is unique the integer sort dominates and CPython's `collections.Counter` — a tight C loop over interned strings — is faster. The number of distinct k-mers is bounded by $\min(N, 4^k)$ for N windows, which supplies a cheap dispatch rule: use the integer path when $4^k \le N$, and otherwise fall back to the string counter. The measured crossover matches this bound (Section 3.4). We regard the guard as part of the method rather than an implementation detail: without it the integer path is a regression over roughly half of the useful range of k.

2.7 Establishing correctness

Each vectorized routine has a straightforward scalar definition, and the repository contains randomized differential tests (`tests/test_equivalence.py`) asserting that the two agree. Generated inputs include ragged batches, empty sequences, ambiguous `N` bases, all three reading frames, both stop-codon policies, k from 1 to 30 spanning both sides of the dispatch rule, and three scoring schemes for alignment. Dynamic programming matrices are compared for exact equality rather than within a tolerance, since the wavefront reorders no arithmetic. These tests run as part of the standard suite.

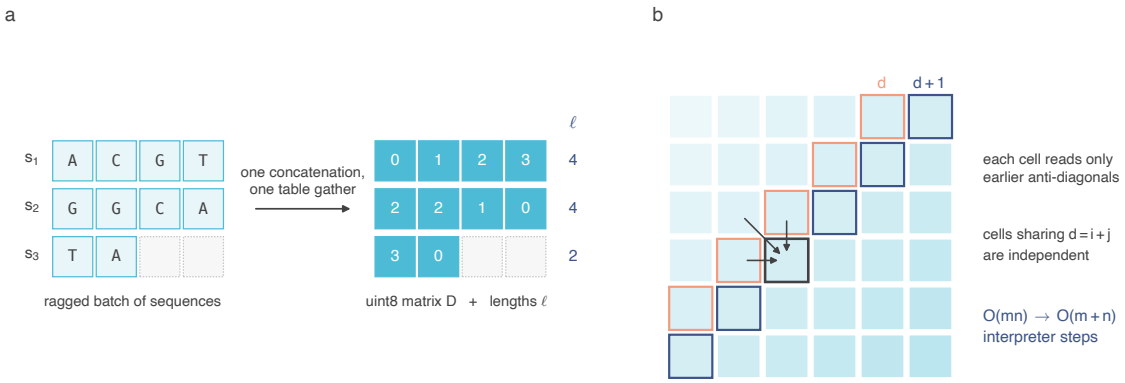

Figure 1: Seqcore's array representation and the wavefront recurrence. **a**, A ragged batch of sequences is stored as one padded matrix D of ordinals together with a vector ℓ of true lengths. Encoding is a single table gather over the concatenated batch, so its cost does not scale with the number of sequences. Dotted cells are padding, excluded by the length mask. **b**, In the alignment matrix, each cell reads only cells on earlier anti-diagonals, so all cells sharing $d = i + j$ are mutually independent and are evaluated in one vector operation. This reduces interpreter-level iteration from $O(mn)$ to $O(m + n)$ without altering the recurrence.

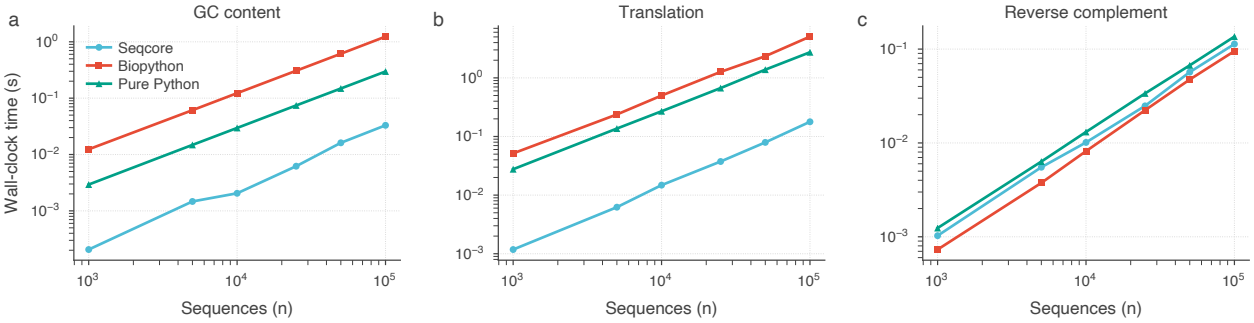

Figure 2: Scaling of batch operations on sequences of 1000 bp, measured on the Mac mini (M4 Pro), shown log–log. All implementations are linear in batch size; what differs is the constant factor. **a**, GC content. **b**, Translation. **c**, Reverse complement, where a single compiled pass (`str.translate` in Biopython) is already close to the memory-bandwidth limit and Seqcore obtains no advantage.

3 Results

3.1 Benchmarking procedure

Measurements were taken on a Mac mini (Apple M4 Pro, 12 cores) running macOS 26.6, arm64, with CPython 3.13.12, NumPy 2.4.3 and Biopython 1.87. We report a desktop-class Apple silicon machine because it is increasingly the hardware on which this kind of analysis is actually run. Each configuration runs in a freshly spawned subprocess, so import order, allocator state and warmed caches cannot leak between implementations, and each reported time is the best of five repetitions (three for alignment and k-mer counting) — the appropriate statistic for throughput on a machine that is not otherwise quiesced. Sequences are uniformly random over $\{A, C, G, T\}$ from a fixed seed.

The pure-Python comparison uses the fastest obvious standard-library formulation of each operation (for instance `str.count` for GC content and `str.translate` for reverse complement), not a deliberately naive one. It is included because it is a meaningful baseline: for several operations it outperforms Biopython, and a library that cannot beat it is not earning its dependency.

The benchmark driver (`benchmarks/benchmark_suite.py`) and its raw JSON output are committed to the repository, and every figure and Table 1 is regenerated from that output by `paper/make_figures.py`. No number in this manuscript is transcribed by hand.

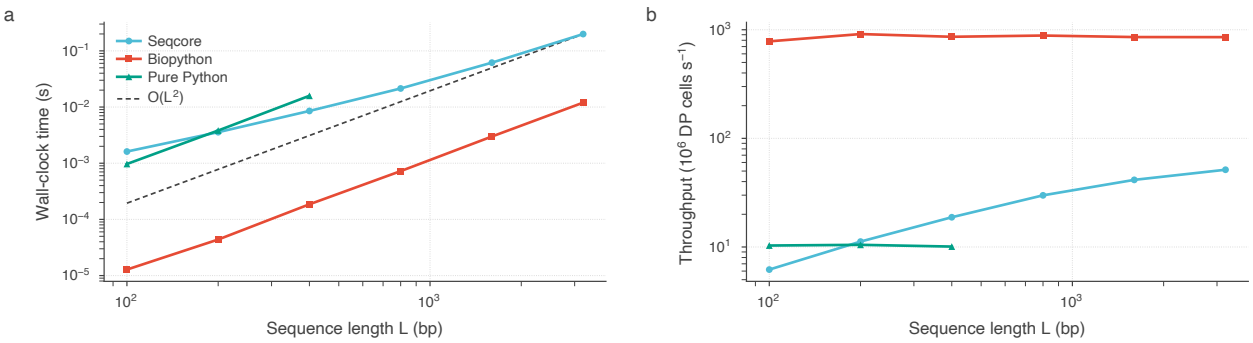

Figure 3: Pairwise global alignment, measured on the Mac mini (M4 Pro). **a**, Wall-clock time against sequence length. Seqcore converges to the $O(L^2)$ reference slope at large L; at small L it is dominated by fixed per-call NumPy overhead. The scalar Python implementation is omitted above 400 bp, where it becomes impractical. **b**, The same data as dynamic-programming cell throughput. Seqcore's throughput rises with L as the fixed cost of each vector operation is amortized over longer anti-diagonals, but plateaus well below Biopython's compiled aligner.

3.2 Batch operations

Figure 2 shows scaling up to 100,000 sequences of 1000 bp. All implementations are linear in batch size, as expected; the separation is in the constant. At 100,000 sequences Seqcore computes GC content in 33 ms and translates in 178 ms, corresponding to single-core throughputs of 3.0 Gbase s^{-1} and 561 Mbase s^{-1}.

Translation shows the largest margin (28.4× Biopython, 15.3× pure Python) because it is the operation where the conventional approach pays the most per residue: a Python-level dictionary lookup and a transient string per codon. Replacing that with an integer index into a lookup table removes both costs at once. GC content behaves similarly, though the absolute margin over pure Python (9.0×) is smaller because `str.count` is itself a compiled scan.

3.3 Pairwise alignment

Figure 3 shows alignment cost against sequence length. Seqcore's cell throughput rises from 6.2 to 51.4 million cells s^{-1} as L grows from 100 to 3200 bp, because the fixed cost of each vector operation is amortized over progressively longer anti-diagonals; at small L the diagonals are short and per-call overhead dominates, which is why the curve in Figure 3a is visibly sub-quadratic there before converging to the $O(L^2)$ reference.

Biopython's aligner sustains roughly 855 million cells s^{-1} across the whole range — flat, as expected for compiled code with no per-call interpreter cost. Seqcore does not approach this and is not intended to; Section 3.6 states the consequence for users.

3.4 k-mer counting

Figure 4a shows the behaviour predicted in Section 2.6. For $k \leq 10$, where 4^k is at or below the window count and k-mers collapse heavily, the integer path is 5.7× faster than the string counter at $k = 8$. Beyond the crossover nearly every window is distinct, the integer path would lose, and the dispatch rule routes those cases to `Counter`; measured times for $k \geq 12$ therefore reflect the fallback and remain 1.8× faster than pure Python through the batched string handling alone. The empirical crossover falls at $k = 12$, exactly where 4^k first exceeds the number of windows.

3.5 Beyond sequence operations

Seqcore also provides modules for small molecules, structural biology, phylogenetics and population genetics. These do not share the sequence array representation, so they are a useful test of how far the paper's argument carries. Figure 5 compares them against the library a user would otherwise reach for.

The bottlenecks there turned out not to be representation at all. Molecular property functions re-parsed the SMILES string on every call, so the cost was repeated work rather than the wrong data structure; caching the parsed molecule removed roughly a factor of forty. Morgan fingerprints spent almost all their

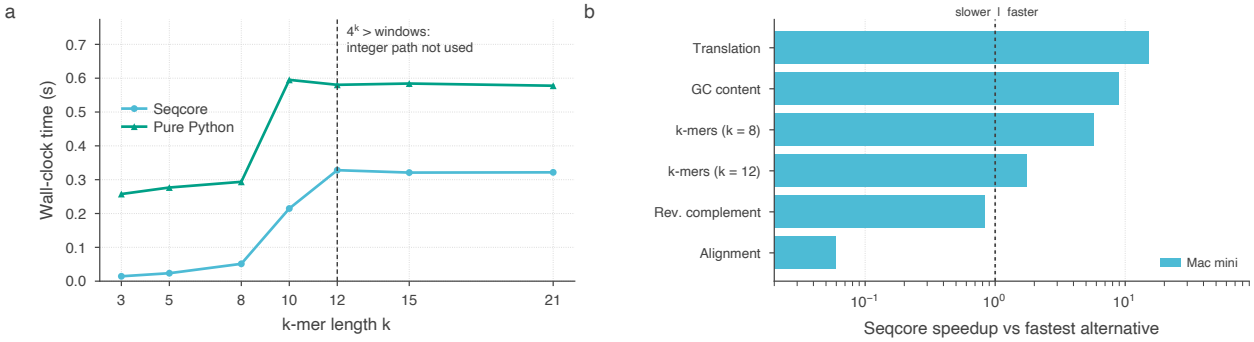

Figure 4: k-mer counting and cross-operation summary. **a**, k-mer counting for 2000 sequences of 1000 bp. The integer-encoding path wins while k-mers collapse; the dashed line marks the $4^k > N$ dispatch rule, beyond which Seqcore falls back to a string counter rather than regressing. Biopython provides no batch k-mer counter and is omitted. **b**, Seqcore's speedup relative to the faster of Biopython and pure Python for each operation, at the configurations in Table 1. Bars left of the dashed line are operations where Seqcore is slower.

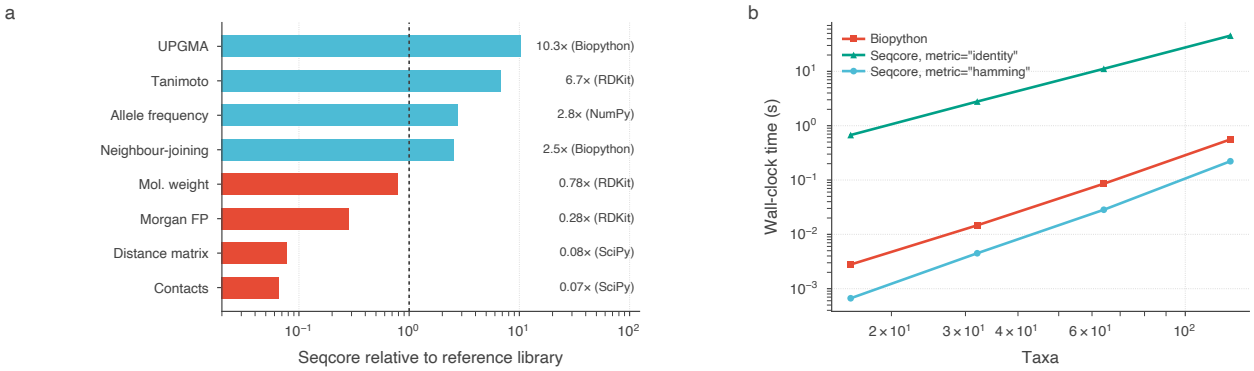

Figure 5: Domain modules against their reference libraries. **a**, Seqcore relative to the library a user would otherwise use for the same operation, at the largest size measured; the reference is named beside each bar. Bars left of the dashed line are operations where the reference is faster. The molecule functions wrap RDKit and the structural searches wrap a spatial index, so parity rather than advantage is the target there. **b**, Neighbour-joining against Biopython. The choice of distance metric dominates: `identity` aligns every pair before building the tree, while `hamming` compares an already-aligned input column by column. Biopython's identity calculator does the latter, so only the lower two curves are a like-for-like comparison.

time converting RDKit bit vectors to arrays one bit at a time in Python. Solvent accessible surface area tested every atom against every other for every surface point, and re-derived van der Waals radii from element strings inside that innermost loop; restricting each atom to the neighbours whose inflated sphere can reach it makes the cost roughly linear rather than quadratic, and at 5000 atoms takes it from tens of minutes to a fraction of a second.

Two results are worth separating from the rest. The molecule functions wrap RDKit, and the structural neighbour searches wrap a spatial index; for those the honest target is parity, not victory, and Figure 5a shows them sitting below 1. A wrapper that costs a few tens of percent buys API consistency at a defensible price; the same wrapper costing seventy times was simply a defect.

Figure 5b makes a different point. For tree building, the user-facing choice of distance metric spans three orders of magnitude — far more than any implementation detail in this paper. `identity` aligns every pair with Needleman–Wunsch, which is correct for unaligned input and costs $O(n^2L^2)$; `hamming` compares an existing alignment column by column. The two are not interchangeable, and disagree once sequences diverge enough for the aligner to open gaps, so the slower and more general option remains the default. We note it because a benchmark that pairs one library's `identity` against another's without checking that they denote the same computation will report a difference that is an artefact of the comparison rather than of either implementation.

3.6 Where Seqcore is not the right tool

Two operations do not favour this design, and we state them plainly rather than omitting them.

Reverse complement runs at 0.84× Biopython (113 ms versus 95 ms at 100,000 sequences). Biopython implements it as `str.translate` followed by a slice: two compiled passes over contiguous bytes, with no padding and no gather indirection. Seqcore performs comparable byte-level work on a padded matrix, and the operation is memory-bandwidth bound rather than dispatch bound, so removing interpreter overhead cannot help. There is no vectorization advantage available here, and we do not claim one.

Pairwise alignment is roughly 17× slower than Biopython's aligner at $L = 3200$. The wavefront still issues $O(m + n)$ NumPy calls, each materializing temporaries proportional to the diagonal length, whereas Biopython dispatches into compiled code with none of that overhead. Vectorized NumPy narrows the gap against scalar Python but cannot close it against C; that requires a compiled kernel. Users whose workload is alignment-bound should use a dedicated aligner, and Seqcore's documentation says so at the call site.

Within the domain modules, `nucleotide_diversity` and `tajimas_d` remain quadratic in the number of sequences, and tree construction itself is still an interpreted $O(n^3)$ loop, which becomes the limiting cost once the distance matrix is no longer the bottleneck.

Two further limitations are worth recording. Seqcore's `align()` accepts a `gap_extend` argument that currently has no effect: gaps are scored linearly using `gap_open` alone, and affine gap penalties are not yet implemented. This is documented in the function's docstring and covered by a regression test. Separately, Seqcore ships CuPy device-management utilities, but no analysis kernel dispatches to the GPU; all Seqcore results in this paper are single-core CPU measurements (see Section 3.7).

3.7 What a GPU backend could and could not deliver

Seqcore does not currently compute on the GPU. Because the representation in Section 2.1 is a plain numeric array, porting the elementwise kernels to CuPy [7] is mechanically straightforward, so it is worth measuring what that port would be worth before undertaking it. We transcribed four Seqcore kernels to CuPy against the identical ordinal matrix, verified that they reproduce the CPU results, and timed them on an NVIDIA A10G (Figure 6). Timings synchronize the CUDA stream inside the measured region, since CuPy launches kernels asynchronously.

The result is a caution against the obvious headline. Measured with the matrix already resident on the device, the kernels are 23–326× faster than Seqcore's CPU path. Measured per call from host memory — which is what a user calling `sc.reverse_complement(arr)` would actually experience — the same kernels are 5–20× faster, because transferring the batch across PCIe costs more than the arithmetic. For reverse complement, which performs one table lookup per byte, transfer accounts for 97% of the elapsed time; the operation with the highest arithmetic intensity, k-mer counting at $k = 8$, retains almost all of its advantage.

Both numbers are correct measurements of different situations, and Figure 6c shows the relationship between them: if the encoded matrix stays on the device across N successive operations, the single transfer is amortized and the effective speedup rises from the per-call value towards the resident-data limit. For reverse complement the crossover is fast — three chained operations already double the benefit — whereas k-mer counting begins near its ceiling.

The design implication is specific. A GPU backend implemented per function, with each call copying in and out, would deliver roughly 5× on the bandwidth-bound operations. Capturing the larger factor requires `DNAArray` itself to hold device memory, transferring only at the I/O boundaries. That is an architectural decision about where the array lives, not a kernel-level optimization, and it is the form the work will take.

We report no GPU performance for Seqcore itself, because there is none to report; Figure 6 characterizes a planned backend, not the shipping library. The transfers use pageable host memory, so the per-call figures are a conservative bound: pinned staging buffers would raise them somewhat without changing the structure of the argument.

4 Discussion

Seqcore's premise is that the representation, not the algorithm, is what usually limits this class of Python library. None of the techniques used here are novel in themselves — masked reductions, lookup tables

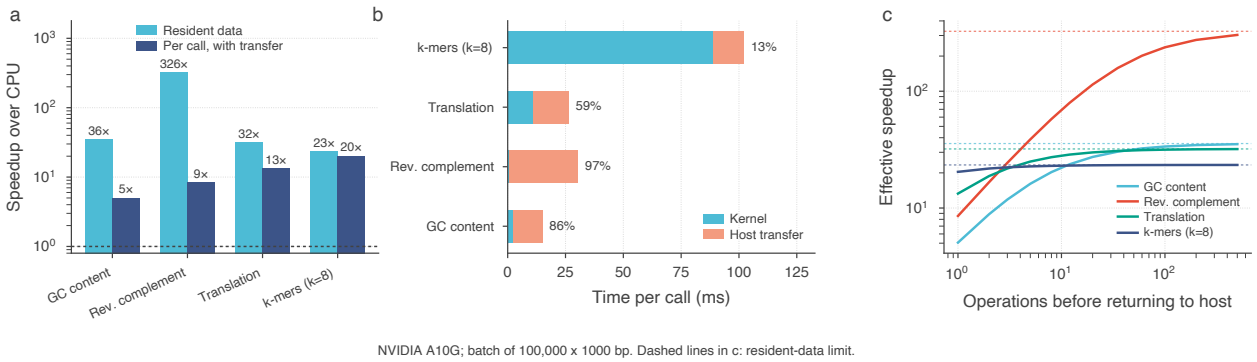

Figure 6: What a GPU backend would be worth. CuPy transcriptions of Seqcore's kernels on an NVIDIA A10G, against Seqcore's CPU path on the same host. **a**, Speedup measured with data already resident on the device (light) and per call including host transfer (dark). **b**, Where the per-call time goes; the annotation gives the transfer share. **c**, Effective speedup when N operations are chained before returning results to the host, so the transfer is paid once; dashed lines mark the resident-data limit from panel **a**. *Seqcore does not currently use the GPU* — these are measurements of a prospective backend, not of the shipping library.

Table 1: Summary of measured performance on the Mac mini (M4 Pro). Times are best-of-N wall clock in seconds. *Speedup* is Seqcore relative to the faster of Biopython and pure Python for that operation; values below 1.0 indicate Seqcore is slower. Biopython provides no batch k-mer counter. This table is generated from the committed benchmark output by `paper/make_figures.py` and shares its source with Figure 4b.

Operation	Configuration	Seqcore (s)	Biopython (s)	Pure Python (s)	Speedup
GC content	100,000 × 1000 bp	0.033	1.237	0.297	9.01×
Translation	100,000 × 1000 bp	0.178	5.058	2.734	15.3×
Reverse complement	100,000 × 1000 bp	0.113	0.095	0.136	0.84×
Global alignment	$L = 3200$ bp	0.199	0.012	—	0.06×
k-mers ($k = 8$)	2,000 × 1000 bp	0.051	—	0.294	5.73×
k-mers ($k = 12$)	2,000 × 1000 bp	0.328	—	0.580	1.77×

and wavefront dynamic programming are all standard — but they only become available once a batch of sequences is a dense numeric array rather than a list of objects. Choosing that representation first is what makes the rest follow.

The domain modules in Section 3.5 qualify that claim rather than confirming it. There the costs were repeated parsing, element-wise conversion at a language boundary, and the absence of a spatial index — none of which is a representation problem. What the two halves share is narrower and perhaps more useful: in each case the measured time was going somewhere other than the arithmetic the user asked for, and the fix was to find where.

Two points generalize beyond this library. The first is that an optimization needs a stated domain of applicability. The k-mer result is the clearest case: the same integer encoding is a large win at small k and a regression at large k, and only the $4^k \leq N$ dispatch rule makes it safe to enable by default. Reporting the mean speedup over a favourable set of k would have hidden this entirely.

The second is that a benchmark is only meaningful if a reader can rerun it. The comparisons here are generated by a committed script whose raw output is also committed, and the figures and table in this paper are produced from that output mechanically. We would encourage this as a default for software papers: it costs little, and it makes the difference between a performance claim and a reproducible measurement.

Finally, we think reporting the unfavourable comparisons is part of the contribution rather than a caveat to it. A library that is 37.5× faster than Biopython at GC content and 17× slower at pairwise alignment is useful, but only to a user who can tell which regime they are in. Presenting only the favourable half would make the library harder to use correctly, not easier.

Future work

The clearest remaining gap is alignment, which needs a compiled kernel rather than further NumPy refinement. The GPU backend analysed in Section 3.7 is the other main direction, and the measurement there sets its design: it must keep the encoded matrix resident on the device rather than transferring per call. The harness used for that analysis (`benchmarks/benchmark_gpu.py`) is included in the repository so the same question can be re-asked on other hardware.

Availability and requirements

Project name: Seqcore
Project home page: `https://github.com/pritampanda15/Seqcore`
Operating systems: Platform independent (tested on Linux, macOS, Windows)
Programming language: Python ≥ 3.9
Other requirements: NumPy ≥ 1.22; optional Biopython, pandas, h5py, RDKit, MDAnalysis, CuPy
Licence: MIT
Version described here: 0.5.0
Installation: `pip install seqcore>=0.5.0`
Everything reported here is reproduced from the repository root with:

```
pytest                                  # correctness and equivalence suite
python benchmarks/benchmark_suite.py    # Figures 2–4, Table 1
python paper/make_figures.py            # regenerate figures and table
```

Competing interests

The author declares no competing interests.

Data availability

All benchmark inputs are synthetic and generated from fixed random seeds by the committed benchmark script. Raw timing output is committed under `benchmarks/results/`.

References

[1] Cock PJA, Antao T, Chang JT, *et al.* Biopython: freely available Python tools for computational molecular biology and bioinformatics. *Bioinformatics* 25(11):1422–1423, 2009.

[2] Harris CR, Millman KJ, van der Walt SJ, *et al.* Array programming with NumPy. *Nature* 585:357–362, 2020.

[3] Needleman SB, Wunsch CD. A general method applicable to the search for similarities in the amino acid sequence of two proteins. *Journal of Molecular Biology* 48(3):443–453, 1970.

[4] Smith TF, Waterman MS. Identification of common molecular subsequences. *Journal of Molecular Biology* 147(1):195–197, 1981.

[5] Wozniak A. Using video-oriented instructions to speed up sequence comparison. *Bioinformatics* 13(2):145–150, 1997.

[6] Rognes T. Faster Smith-Waterman database searches with inter-sequence SIMD parallelisation. *BMC Bioinformatics* 12:221, 2011.

[7] Okuta R, Unno Y, Nishino D, Hido S, Loomis C. CuPy: A NumPy-compatible library for NVIDIA GPU calculations. *Proceedings of the Workshop on Machine Learning Systems (LearningSys) at NIPS*, 2017.